

Comprehensive Technical Documentation

Enterprise Microservices Backend — Architecture, APIs, Security, Resilience Patterns & Deployment

Zomato / Swiggy-style system built with Java 17 and Spring Boot 3

Project: FoodFlow — Food Delivery Backend

Architecture: Microservices (Spring Cloud)

Language: Java 17

Framework: Spring Boot 3.x

Services: 12 (11 microservices + common-lib)

Stack: Spring Cloud, Eureka, JWT, MySQL, OpenFeign, Resilience4j

Table of Contents

1. Executive Summary
2. Technology Stack
3. System Architecture
4. Microservices Overview
5. Layered Architecture
6. Project Structure
7. Security and JWT Authentication
8. Domain Entities and DTOs
9. Inter-Service Communication (Feign)
10. Business Flows
11. REST API Reference
12. Exception Handling
13. Docker Deployment
14. Resilience Patterns in Microservices
15. Circuit Breaker Pattern
16. Resilience4j Configuration
17. Monitoring and Best Practices
18. Setup and Deployment Guide

1. Executive Summary

FoodFlow is an enterprise-grade Food Delivery Microservices Backend similar in concept to Zomato or Swiggy. It is organized as a Maven multi-module monorepo with eleven deployable Spring Boot applications plus a shared common library.

Key design principles:

- Microservices architecture — each service owns its database.
- Layered architecture — controller, service, repository in every service.
- DTO-only APIs — JPA entities are never exposed over HTTP or Feign.
- Eureka service discovery and Spring Cloud Config for centralized settings.
- API Gateway as a single entry point with load-balanced routing.
- JWT authentication with role-based access control (RBAC).
- Resilience patterns using Resilience4j for fault tolerance.

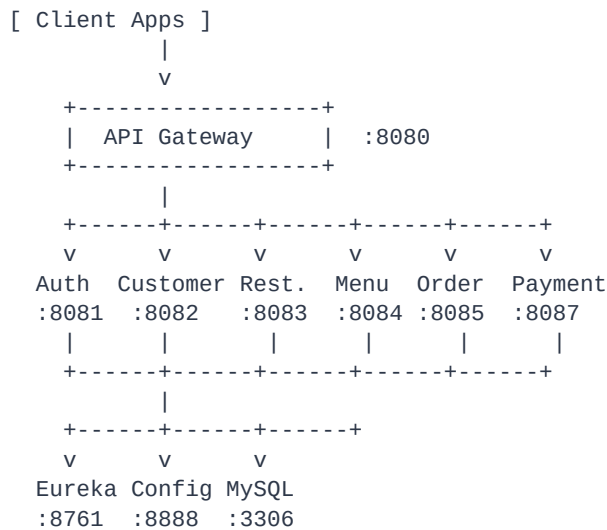
2. Technology Stack

Category	Technology	Version
Language	Java	17
Framework	Spring Boot	3.3.5
Cloud	Spring Cloud	2023.0.3
Database	MySQL + Spring Data JPA	Per service
Security	Spring Security + JWT	jjwt 0.12.6
Inter-service	OpenFeign	order-service
Discovery	Netflix Eureka	Port 8761
Config	Spring Cloud Config	Port 8888
Gateway	Spring Cloud Gateway	Port 8080
Resilience	Resilience4j	2.1.0
API Docs	SpringDoc OpenAPI	Swagger UI
Build	Maven	Multi-module

3. System Architecture

Clients connect only to the API Gateway (port 8080). The gateway uses Eureka to find service instances and routes requests. The Order Service orchestrates other services using OpenFeign when a customer places an order.

3.1 Architecture Diagram



3.2 Service Ports and Databases

Service	Port	Database	Role
discovery-server	8761	-	Eureka registry
config-server	8888	-	Central configuration
api-gateway	8080	-	API routing
auth-service	8081	auth_db	JWT login/register
customer-service	8082	customer_db	Customer CRUD
restaurant-service	8083	restaurant_db	Restaurants
menu-service	8084	menu_db	Menu items
order-service	8085	order_db	Order orchestration
delivery-partner-service	8086	delivery_db	Delivery
payment-service	8087	payment_db	Payments
notification-service	8088	notification_db	Alerts

4. Microservices Overview

4.1 Infrastructure Services

Discovery Server: All services register on startup. Dashboard at <http://localhost:8761>

Config Server: Reads YAML from config-repo/. Each service imports config via `spring.config.import`.

API Gateway: Routes `/api/customers/**` to `customer-service`, etc. Uses `StripPrefix=1` so `/api/customers/1` becomes `/customers/1` downstream.

4.2 Common Library (common-lib)

- Enums: `Role`, `OrderStatus`, `DeliveryStatus`, `PaymentStatus`
- Exceptions: `ResourceNotFoundException`, `InvalidRequestException`, `UnauthorizedException`
- `GlobalExceptionHandler` with uniform `ApiResponse`

5. Layered Architecture (Every Service)

```
controller      -> REST endpoints, ResponseEntity, @Valid
  |
  service        -> Interface (business contract)
  |
  serviceImpl    -> Business logic implementation
  |
  repository     -> Spring Data JPA
  |
  entity         -> @Entity (NEVER exposed in API)

Also: dto/, mapper/, config/, security/, exception/, client/
```

Rule: Controllers use `RequestDto` and `ResponseDto` only. Mappers convert between `Entity` and `DTO` inside the service layer.

6. Project Structure

```
food-delivery-platform/
|-- pom.xml                (parent)
|-- common-lib/            (shared code)
|-- config-repo/           (YAML configs)
|-- discovery-server/
|-- config-server/
|-- api-gateway/
|-- auth-service/
|-- customer-service/
|-- restaurant-service/
|-- menu-service/
|-- order-service/         (Feign clients in client/)
|-- delivery-partner-service/
|-- payment-service/
|-- notification-service/
|-- docker-compose.yml
|-- Dockerfile
```

7. Security and JWT Authentication

Role	Purpose
ADMIN	Platform administration
CUSTOMER	Place orders (referenceId = customerId)
RESTAURANT_OWNER	Manage restaurants and menus
DELIVERY_PARTNER	Update delivery status

7.1 Login Flow

- POST /auth/register with email, password, role, referenceId
- Password stored with BCryptPasswordEncoder
- POST /auth/login returns JWT token
- Send header: Authorization: Bearer
- JwtAuthenticationFilter validates token on each request

7.2 JWT Token Contract

Claims:

```
sub      = user email (String)
role     = "ROLE_CUSTOMER" | "ROLE_ADMIN" | "ROLE_RESTAURANT_OWNER" | "ROLE_DELIVERY_PARTNER"
userId   = Long (maps to auth_users table)
referenceId = Long (customerId / restaurantId / partnerId – 0 for ADMIN)
iat      = issued at
exp      = expiry (configurable, default 24h)
```

Header forwarded by Gateway:

```
X-Auth-User-Id      : userId value
X-Auth-User-Role    : role value
X-Auth-Reference-Id : referenceId value
```

8. Domain Entities and DTOs

Entity	Service	Main Fields
User	auth	email, password, role, referenceId
Customer	customer	name, email, phone, address
Restaurant	restaurant	restaurantName, ownerName, address, rating
MenuItem	menu	restaurantId, itemName, price, availability
Order	order	customerId, restaurantId, totalAmount, orderStatus
DeliveryPartner	delivery	name, phone, vehicleNumber, availabilityStatus
Payment	payment	orderId, amount, paymentStatus, paymentMethod
Notification	notification	userId, message, type, sentAt

8.1 Status Enums

Enum	Values
OrderStatus	PLACED, CONFIRMED, PREPARING, READY_FOR_PICKUP, PICKED_UP, OUT_FOR_DELIVERY
DeliveryStatus	ASSIGNED, PICKED_UP, OUT_FOR_DELIVERY, DELIVERED, DELIVERY_FAILED, REASSIGNED
PaymentStatus	PENDING, SUCCESS, FAILED, REFUNDED

9. Inter-Service Communication (OpenFeign)

Order service orchestrates other services using Feign. All calls use DTOs in package `client/` — never entities.

Feign Client	Target	Endpoint
CustomerServiceClient	customer-service	GET /api/v1/customers/{id}
RestaurantServiceClient	restaurant-service	GET /api/v1/restaurants/{id}
MenuServiceClient	menu-service	POST /api/v1/menu/validate
PaymentServiceClient	payment-service	POST /api/v1/payments
NotificationServiceClient	notification-service	POST /api/v1/notifications
DeliveryServiceClient	delivery-partner-service	POST /api/v1/delivery-partners/assign

10. Business Flows

10.1 Customer Registration and Login

Step 1: POST /api/customers -> Create customer (get id)
Step 2: POST /api/auth/register -> role=CUSTOMER, referenceId=<id>
Step 3: POST /api/auth/login -> Get JWT token
Step 4: Use Bearer token on API calls

10.2 Restaurant and Menu

Step 1: Register RESTAURANT_OWNER in auth-service
Step 2: POST /api/restaurants -> Create restaurant
Step 3: POST /api/menus -> Add menu items
Step 4: PUT /api/menus/{id} -> Update availability

10.3 Order Placement (Orchestration)

POST /api/orders

1. Feign: validate customer exists (customer-service)
2. Feign: validate restaurant exists (restaurant-service)
3. Feign: validate menu items (menu-service)
4. Save order (status = PLACED)
5. Feign: process payment (payment-service) -> SUCCESS/FAILED
6. On SUCCESS: keep status PLACED, trigger notification
7. On FAILED: set status CANCELLED, trigger notification
8. Feign: send notification (notification-service)

10.4 Delivery Flow

POST /api/orders/{id}/assign-delivery
-> Assigns nearest available partner (status = ASSIGNED)
-> Order status = OUT_FOR_DELIVERY

PATCH /api/delivery-partners/{id}/status
-> ASSIGNED -> PICKED_UP -> OUT_FOR_DELIVERY -> DELIVERED
-> Or DELIVERY_FAILED -> REASSIGNED

11. REST API Reference

Base URL (via Gateway): **http://localhost:8080/api/v1**

11.1 Auth Service

Method	Endpoint	Description
POST	/api/v1/auth/register	Register with role
POST	/api/v1/auth/login	Login, get JWT

11.2 Customer Service

Method	Endpoint	Description
POST	/api/v1/customers	Create customer
GET	/api/v1/customers/{id}	Get by ID
GET	/api/v1/customers	List all (paginated)
PUT	/api/v1/customers/{id}	Update
DELETE	/api/v1/customers/{id}	Delete

11.3 Restaurant Service

Method	Endpoint	Description
POST	/api/v1/restaurants	Create
GET	/api/v1/restaurants/{id}	Get by ID
GET	/api/v1/restaurants	List all
PUT	/api/v1/restaurants/{id}	Update
DELETE	/api/v1/restaurants/{id}	Delete

11.4 Menu Service

Method	Endpoint	Description
POST	/api/v1/menus	Create menu item
GET	/api/v1/menus/{id}	Get item
GET	/api/v1/menus/restaurant/{id}	Menu by restaurant
PUT	/api/v1/menus/{id}	Update
DELETE	/api/v1/menus/{id}	Delete

11.5 Order Service

Method	Endpoint	Description
POST	/api/v1/orders	Place order
GET	/api/v1/orders/{id}	Get order
GET	/api/v1/orders/customer/{id}	By customer
PATCH	/api/v1/orders/{id}/status	Update status
POST	/api/v1/orders/{id}/assign-delivery	Assign partner

11.6 Universal Response Envelope

```
// Success
{
  "success": true,
  "message": "Customer retrieved successfully",
  "data": { ... },
  "timestamp": "2024-01-15T10:30:00Z"
}

// Error
{
  "success": false,
  "message": "Customer not found with id: 42",
  "data": null,
  "timestamp": "2024-01-15T10:30:00Z",
  "errors": ["field: validation message"]
}
```

12. Exception Handling

Exception	HTTP Status
ResourceNotFoundException	404 Not Found
InvalidRequestException	400 Bad Request
UnauthorizedException	403 Forbidden
MethodArgumentNotValidException	400 with field errors
Exception (catch-all)	500 Internal Server Error

13. Docker Deployment

Docker Compose runs the entire platform in containers, avoiding port conflicts. Only ports 8080, 8761, 8888, and 3307 are exposed on the host.

13.1 Container Overview

Container	Host Port	Internal Only
fd-mysql	3307	No
fd-discovery	8761	No
fd-config	8888	No
fd-gateway	8080	No
fd-auth	-	Yes
fd-customer	-	Yes
fd-restaurant	-	Yes
fd-menu	-	Yes
fd-order	-	Yes
fd-delivery	-	Yes
fd-payment	-	Yes
fd-notification	-	Yes

13.2 Quick Start

```
chmod +x docker-start.sh
./docker-start.sh
```

Or manually:

```
docker compose build
docker compose up -d
```

13.3 Startup Order

- MySQL (healthy check)
- Eureka discovery-server (healthy)
- Config server (healthy, mounts config-repo/)
- All business services (parallel)
- API Gateway (last)

13.4 Useful Commands

```
docker compose ps           # Status of all containers
docker compose logs -f order-service # Logs for one service
docker compose restart order-service # Restart one service
```

```
docker compose down                                # Stop everything
docker compose down -v                             # Stop and remove database volume
```

14. Resilience Patterns in Microservices

Resilience is the ability of a system to continue functioning correctly even when some of its components fail or experience degraded performance.

14.1 What is Resilience

- Services are distributed and communicate over the network
- Each service has its own failure domain
- A single service failure can cascade to dependent services
- Network latency and timeouts are inherent in distributed systems

14.2 Problems Caused by Cascading Failures

Cascading failures occur when a failure in one service triggers failures in dependent services, creating a domino effect that can bring down the entire system.

Example Scenario:

```
Payment Service slows down → Order Service threads block waiting
→ Order Service thread pool exhausts → API Gateway requests timeout
→ Customer retries → More requests flood the system
→ All services become unresponsive → System-wide outage
```

14.3 When a Dependent Service is Slow or Down

Condition	Impact
Service DOWN	Connection timeouts, Feign throws <code>ConnectException</code> , threads block indefinitely
Service SLOW	High latency, backpressure builds up, resource pools exhausted

15. Circuit Breaker Pattern

The Circuit Breaker pattern prevents cascading failures by detecting when a dependent service is failing and temporarily blocking calls to it.

15.1 How It Works

- Monitor calls to a dependent service
- Count failures over a sliding time window
- When failure threshold is reached, trip the circuit (OPEN state)
- Block all calls immediately (fail fast)
- After wait duration, attempt a single call (HALF-OPEN state)
- If successful, reset to CLOSED; if failed, stay OPEN

15.2 Circuit Breaker States

State	Description
CLOSED	Normal operation - all requests pass through, failure rate monitored
OPEN	Circuit tripped - all requests blocked, fallback invoked, wait timer starts
HALF-OPEN	Recovery attempt - single test call allowed to check if service recovered

15.3 Retry vs Circuit Breaker

Aspect	Retry Pattern	Circuit Breaker Pattern
Purpose	Handle transient failures	Prevent cascading failures
When to use	Network glitches, temporary timeouts	Service down, persistent failures
Behavior	Re-attempt the same request	Block requests to failing service
Resource usage	Increases load (more requests)	Decreases load (blocks requests)

16. Resilience4j Configuration

Resilience4j is a fault tolerance library for Java that implements several resilience patterns.

16.1 Patterns Provided

- Circuit Breaker: Prevent cascading failures
- Retry: Handle transient failures
- Rate Limiter: Control request rate
- Bulkhead: Limit concurrent calls
- Time Limiter: Cancel long-running operations
- Cache: Cache responses to reduce load

16.2 Circuit Breaker Configuration

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        slidingWindowSize: 10           # Number of calls in sliding window
        minimumNumberOfCalls: 5         # Minimum calls before calculating rate
        failureRateThreshold: 50        # Failure percentage to trip (50%)
        waitDurationInOpenState: 60s    # Time in OPEN before HALF-OPEN
        permittedNumberOfCallsInHalfOpenState: 3 # Test calls in HALF-OPEN
        slowCallRateThreshold: 100      # Slow call rate threshold
        slowCallDurationThreshold: 2s   # What qualifies as "slow"
```

16.3 Implementing Fallback Methods

```
@Component
@Slf4j
public class PaymentServiceClientFallback implements PaymentServiceClient {
    @Override
    public ApiResponse<PaymentResponseDto> createPayment(PaymentRequestDto dto) {
        log.error("Payment service unavailable, using fallback");
        return ApiResponse.<PaymentResponseDto>error(
            "Payment service temporarily unavailable. Please try again later.",
            List.of("PAYMENT_SERVICE_UNAVAILABLE")
        );
    }
}
```

17. Monitoring and Best Practices

17.1 Circuit Breaker Metrics

- Circuit state (CLOSED, OPEN, HALF-OPEN)
- Failure rate
- Success rate
- Number of buffered calls
- Number of failed calls
- Number of slow calls

17.2 Actuator Endpoints

```
GET /actuator/circuitbreakers
GET /actuator/metrics/resilience4j.circuitbreaker.state
GET /actuator/metrics/resilience4j.circuitbreaker.failure.rate
GET /actuator/prometheus
```

17.3 Common Configuration Mistakes

Mistake	Impact	Fix
Failure threshold too high (90%)	Too many failures before circuit opens	Use 50-60% threshold
Wait duration too short (5s)	Circuit flips rapidly	Use 30-60 seconds minimum
No timeout on Feign clients	Threads block indefinitely	Always set connect and read timeouts
Retry without exponential backoff	Retry storms overwhelm service	Use exponential backoff
Circuit breaker on non-critical paths	Unnecessary complexity	Use fallback only

17.4 When NOT to Use Retry

- Non-idempotent operations (POST without idempotency key)
- Business logic failures (insufficient funds, invalid data)
- Authentication failures (wrong password)
- Rate limit exceeded (429 status)
- Validation errors (400 status)

17.5 Best Practices Summary

- Use Circuit Breaker for all cross-service calls
- Use Retry only for transient network failures
- Always configure timeouts on Feign clients
- Implement meaningful fallbacks (not just null returns)
- Monitor circuit state and metrics
- Test failure scenarios in integration tests

- Don't over-engineer for non-critical paths

18. Setup and Deployment Guide

18.1 Prerequisites

- Java 17+, Maven 3.8+, MySQL 8
- Docker Engine 20+, Docker Compose v2 (for Docker deployment)
- 8 GB+ RAM recommended (11 JVMs for Docker)

18.2 Local Maven Startup

```
# Start MySQL
docker compose up -d mysql

# Build project
./mvnw clean install -DskipTests

# Start services (order matters)
./mvnw -pl discovery-server spring-boot:run
./mvnw -pl config-server spring-boot:run
./mvnw -pl auth-service, customer-service, restaurant-service, menu-service, payment-service, notification-service spring-boot:run
./mvnw -pl api-gateway spring-boot:run
```

18.3 Access Points

Service	URL
API Gateway	http://localhost:8080
Eureka Dashboard	http://localhost:8761
Config Server	http://localhost:8888
Swagger UI (per service)	http://localhost:{port}/swagger-ui.html

18.4 Sample cURL Commands

```
# Create customer
curl -X POST http://localhost:8080/api/v1/customers \
  -H "Content-Type: application/json" \
  -d '{"name":"John","email":"john@mail.com","phone":"9876543210","address":"Mumbai"}'

# Register auth
curl -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"john@mail.com","password":"secret1","role":"CUSTOMER","referenceId":1}'

# Place order
curl -X POST http://localhost:8080/api/v1/orders \
  -H "Content-Type: application/json" \
  -d '{"customerId":1,"restaurantId":1,"totalAmount":499.0,"paymentMethod":"UPI"}'
```